ShelfQuery — Hints

> Read the story and try to build it first. These hints point you at the pieces and the easy-to-miss parts, without giving the whole answer. The full code is in the complete solution.

Hint 1 — the pieces, and where to start

The same shape as SQLSage, for a library catalog:

- `src/guard.py` — `is_safe_select(sql)` : `True` only for a single read-only `SELECT` . **Build this first** — it's plain Python, no model, so you can test the whole safety story with hand-written SQL.
- `src/db/seed.py` — `build_sample_db(db_path)` : create `books` and `loans` , insert a few rows (only if the file doesn't exist). `src/db/schema.py` — `schema_text(db_path)` : the `CREATE TABLE` statements as one string. `src/db/runner.py` — `run_select(db_path, sql)` : execute and return rows.
- `src/schemas.py` — `SQLQuery(sql)` (Pydantic). `src/sql_agent.py` — `generate_sql` with `model.with_structured_output(SQLQuery)` , schema in the prompt.
- `src/answer.py` — `summarize_answer(model, question, rows)` : one sentence from the rows only.
- `src/pipeline.py` — `ask` : schema → generate → **guard** → run → summarize.

Build order: guard + database (test them) → generate_sql → summarize_answer → pipeline.

Hint 2 — the guard, and the order in the pipeline

The guard is the safety of the whole project:

```
is_safe_select(sql):
    cleaned = sql.strip().rstrip(";").strip()   # allow one trailing ;
    if not cleaned.lower().startswith("select"): return False
    if ";" in cleaned: return False             # no chained statements
    for word in FORBIDDEN_WORDS: ...            # insert/update/delete/drop/...
    return True
```

Two easy-to-miss parts: **strip one trailing semicolon** but **reject any remaining `;`** so `SELECT ...; DROP TABLE ...` can't slip through; and keep the forbidden list lowercase and compare

against the lowercased query.

In the pipeline, the order is the point — the guard sits **between** generation and execution:

```
sql = generate_sql(model, schema, question)
if not is_safe_select(sql):
    raise UnsafeQueryError(sql)     # nothing unsafe reaches the catalog
rows = run_select(db_path, sql)
```

Keep the model at `temperature=0` so the SQL and the phrasing are repeatable. The whole file set is in the complete solution.